

Übersetzung und Analyse von Programmen

The background is a stylized, low-poly illustration. It features a multi-story building with large windows in shades of blue and green. A large, bright yellow sun is in the upper right. In the foreground, there are two green trees with yellow leaves and a large, detailed yellow flower with a dark center. The overall color palette is dominated by blues, greens, and yellows.

Vorlesung im Wintersemester 2025/26

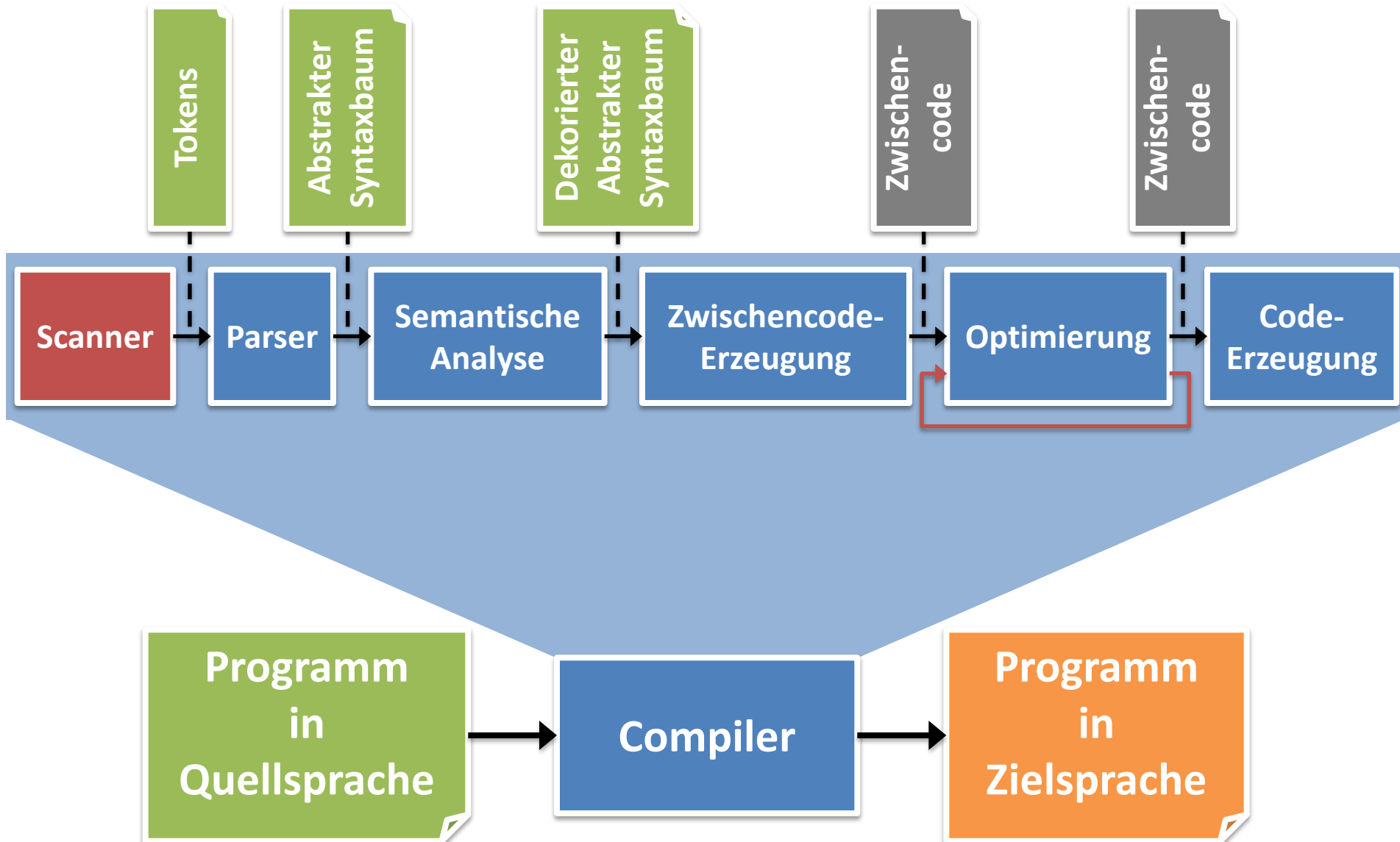
Prof. Dr. Stephan Diehl

Informatik, Universität Trier



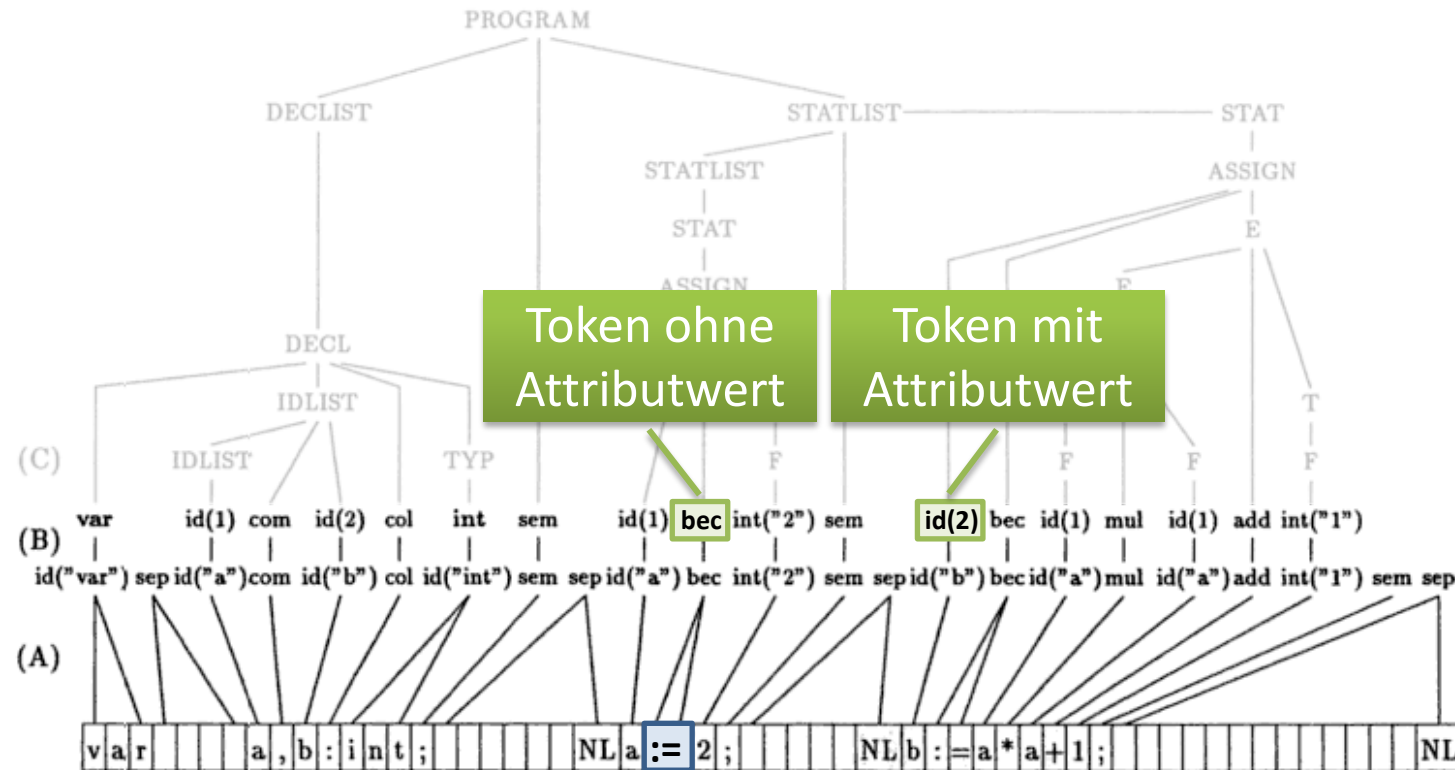
- Lexikalische Analyse
 - Zerlegung des Quelltexts in logische Einheiten
 - Reguläre Ausdrücke, endliche Automaten
 - Generierung eines Scanners

Struktur eines Compilers



Struktur eines Compilers

Scanner



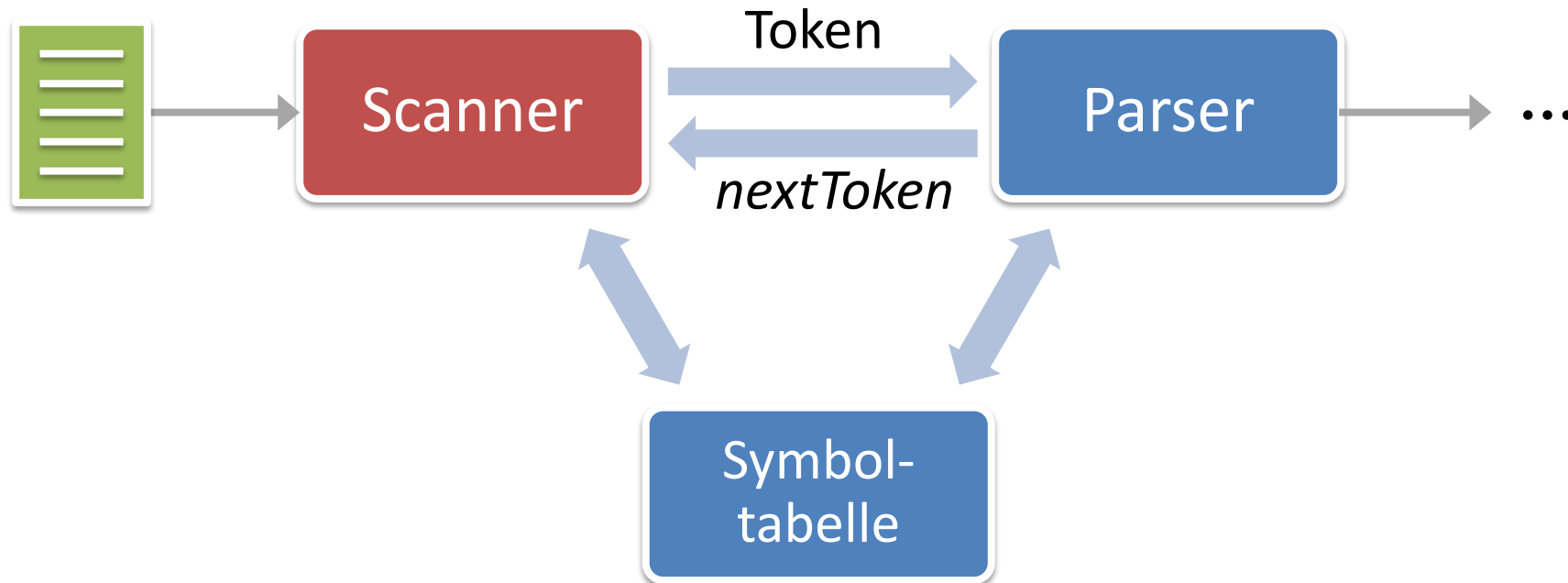
Quelle: Übersetzerbau, Wilhelm & Maurer, 1992

- (A) Lexikalische Analyse
- (B) Filtern/Sieben
- (C) Syntaktische Analyse

Lexem

Muster

Zusammenspiel zwischen Scanner und Parser



Symboltabelle

- Speicherung von Informationen über die im Programm verwendeten Bezeichner
 - String/Lexem, Typ, Speicherort, ...
- Informationen werden während der Analysephase (Frontend) schrittweise ergänzt
- Je eine Tabelle pro Gültigkeitsbereich

Beispiel:

a	int	
b	int	

Worte und Sprachen

Worte und Sprachen

- Ein **Alphabet** Σ ist eine endliche, nichtleere Menge von Zeichen.
- Ein **Wort** x über Σ der Länge n ist eine endliche Folge von Zeichen aus Σ :
 $x : \{1, \dots, n\} \rightarrow \Sigma$ in Zeichen: $x_1 x_2 \dots x_n$
- Das **leere Wort** ε ist das Wort, das aus keinem Zeichen besteht: $\varepsilon : \emptyset \rightarrow \Sigma$
- Menge des leeren Wortes: $\Sigma^0 = \{\varepsilon\}$
- Menge der Worte der Länge n : $\Sigma^n = \{x \mid x : \{1, \dots, n\} \rightarrow \Sigma\}$
- Menge aller Worte über Σ : $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$
- Menge aller nichtleeren Worte über Σ : $\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i$

Worte und Sprachen

- $x.y$ ist die **Konkatenation** von x und y .
$$x.y = x_1 \dots x_n y_1 \dots y_m \quad \text{mit } x = x_1 \dots x_n \text{ und } y = y_1 \dots y_m$$
- **Bemerkungen**
 - Das leere Wort ist **neutrales Element** bezüglich der Konkatenation von Worten: $x.\varepsilon = \varepsilon.x = x$ für alle $x \in \Sigma^*$
 - Die Konkatenation ist **assoziativ**: $x.(y.z) = (x.y).z$
 - Wir schreiben im folgenden xy für $x.y$

Weitere Begriffe

Sei $w = xyz$ mit $x, y, z \in \Sigma^*$. Dann ist

- x ein **Präfix** von w
 - Falls $yz \neq \varepsilon \neq x$ gilt, ist x ein **echter Präfix** von w
- z ein **Suffix** von w
 - Falls $xy \neq \varepsilon \neq z$ gilt, ist z ein **echter Suffix** von w
- y ein **Teilwort** von w
 - Falls $xz \neq \varepsilon \neq y$ gilt, ist y ein **echtes Teilwort** von w

Operationen auf formalen Sprachen

Formale **Sprachen** über Σ sind Teilmengen von Σ^* .

Seien $L, L_1, L_2 \dots$ formale Sprachen

- Vereinigung von Sprachen: $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ oder } w \in L_2\}$
- Konkatenation von Sprachen: $L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\}$
- Komplement einer Sprache: $\neg L = \Sigma^* - L$
- Potenz einer Sprache: $L^0 = \{\varepsilon\}, L^{n+1} = L^n L$
- Abschluss (Hülle) einer Sprache: $L^* = \bigcup_{i=0}^{\infty} L^i$
- Positiver Abschluss (Hülle) einer Sprache: $L^+ = \bigcup_{i=1}^{\infty} L^i$

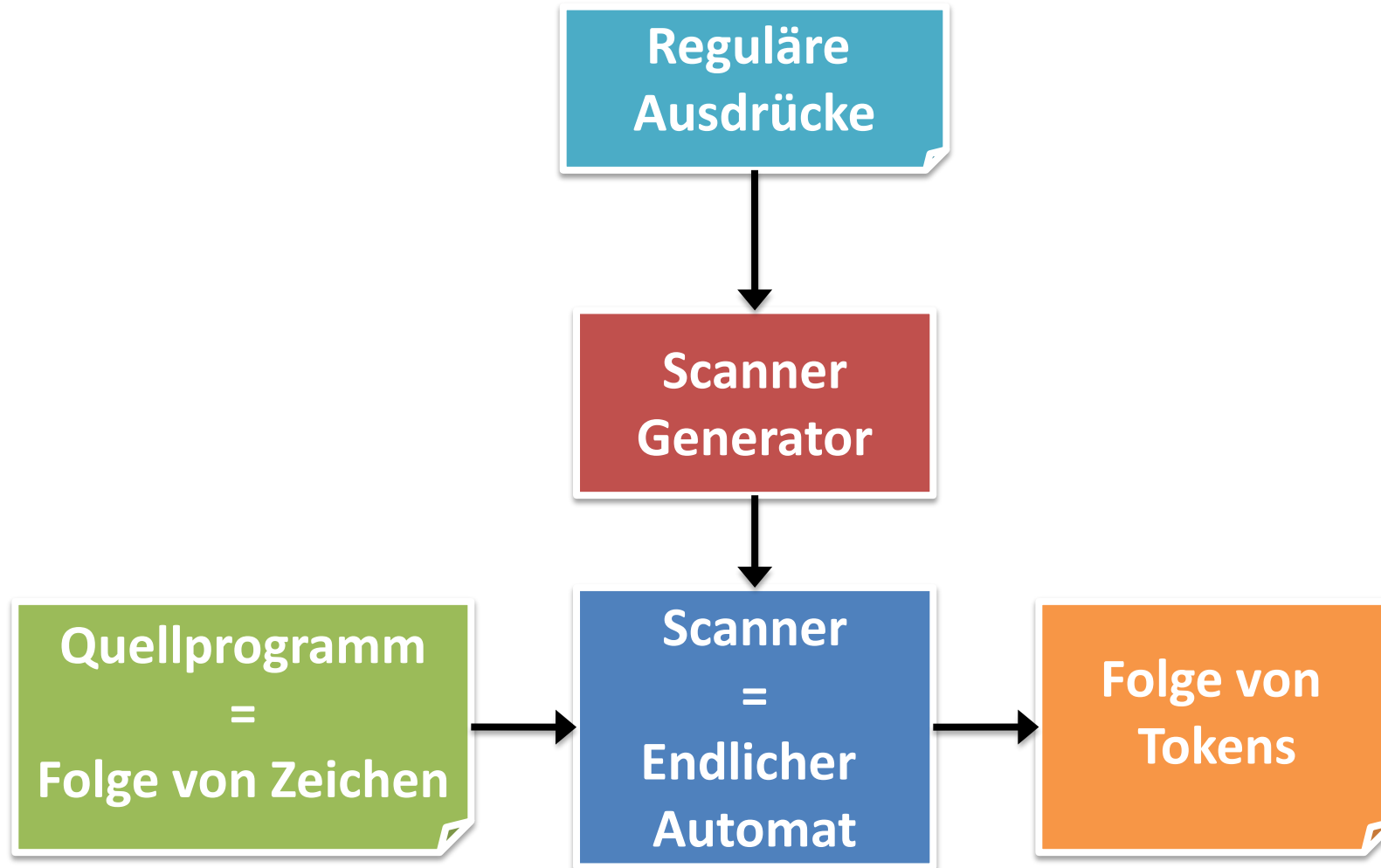
Definition: Reguläre Sprache

- Die **regulären Sprachen** über Σ werden induktiv definiert durch:
 - $\emptyset, \{\varepsilon\}$ sind reguläre Sprachen über Σ .
 - Für alle $a \in \Sigma$ ist $\{a\}$ eine reguläre Sprache.
 - Sind R_1 und R_2 reguläre Sprachen über Σ , so auch
 - $R_1 \cup R_2$,
 - $R_1 R_2$,
 - und R_1^* bzw. R_2^*
 - Nichts sonst ist eine reguläre Sprache über Σ .

Lexikalische Analyse

- **Reguläre Sprachen** können durch **reguläre Ausdrücke** beschrieben werden.
- Deshalb bilden diese die Basis für alle Sprachen zur Spezifikation der **lexikalischen Analyse**.
- Reguläre Sprachen können durch **endliche Automaten** erkannt werden.

Lexikalische Analyse



Reguläre Ausdrücke

- Betrachte als Beispiel Bezeichner, wie etwa **begin**, **end**, **xyz123**, **y2**
- Natürlichsprachliche Spezifikation:
 - *Bezeichner bestehen aus einer Aneinanderreihung von Buchstaben und Ziffern, wobei das erste Zeichen ein Buchstabe sein muss.*
- Kürzer: ein **BUCHSTABE**, dann beliebig oft (ein **BUCHSTABE** oder eine **ZIFFER**)
 - **DANN** entspricht der Konkatenation zweier Worte,
 - **ODER** entspricht der Alternative zwischen zwei Worten (in Zeichen: |),
 - **BELIEBIG OFT** entspricht dem Kleene-Stern (in Zeichen: *).
- Als regulärer Ausdruck: **bu (bu | zi)***
 - Dies ist ein regulärer Ausdruck wobei **bu** und **zi** auch wieder reguläre Ausdrücke sind
 - bu** = **a | b | c ... | y | z**
 - zi** = **0 | 1 | ... | 9**

Definition: regulärer Ausdruck

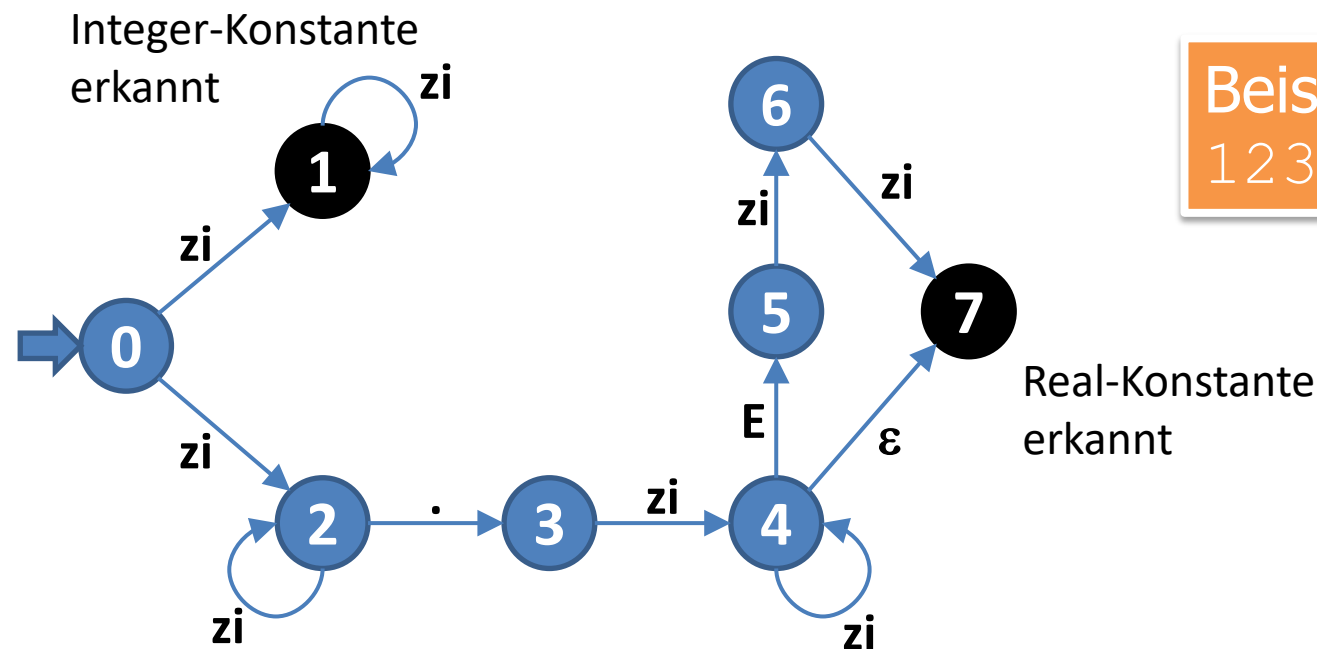
Reguläre Ausdrücke (RA) und die von ihnen beschriebenen regulären Sprachen lassen sich ebenfalls induktiv definieren:

- $\emptyset$ ist ein regulärer Ausdruck über Σ und beschreibt die reguläre Sprache $\emptyset$.
- ε ist ein regulärer Ausdruck über Σ und beschreibt die reguläre Sprache $\{\varepsilon\}$.
- a (für $a \in \Sigma$) ist ein regulärer Ausdruck über Σ und beschreibt die reguläre Sprache $\{a\}$.
- Sind r_1 und r_2 reguläre Ausdrücke, welche die regulären Sprachen R_1 bzw. R_2 beschreiben, so ist:
 - $(r_1 \mid r_2)$ ein regulärer Ausdruck und beschreibt die reguläre Sprache $R_1 \cup R_2$,
 - $(r_1 r_2)$ ein regulärer Ausdruck und beschreibt die reguläre Sprache $R_1 R_2$,
 - $(r_1)^*$ ein regulärer Ausdruck und beschreibt die reguläre Sprache R_1^* .
- Nichts sonst ist ein regulärer Ausdruck.

Übergangsdiagramme

Beispiel: regulärer Ausdruck, der Integer- und Realkonstanten ohne Vorzeichen:

$((\text{zi} (\text{zi})^*) \mid (\text{zi} (\text{zi})^* . \text{zi} (\text{zi})^* (\text{E} \text{zi} \text{zi} \mid \varepsilon)))$



Beispiel:
123.45E12

Die von dem Übergangsdiagramm akzeptierte Sprache ist die Menge der Wörter, für die es einen Weg von einem Startknoten zu einem Endknoten gibt.

Definition: Übergangsdiagramm

Ein **Übergangsdiagramm** ist ein endlicher, gerichteter, kantenmarkierter Graph

$$UD = (V, E, T, v_0, V_f), \text{ wobei}$$

- ② – V eine endliche Menge von Knoten,
- zi → – E eine endliche Menge von markierten Kanten,
- ① – $v_0 \in V$ ein ausgezeichneteter Knoten, der **Startknoten**,
- ① – und $V_f \subseteq V$ die Menge der Endknoten sind.

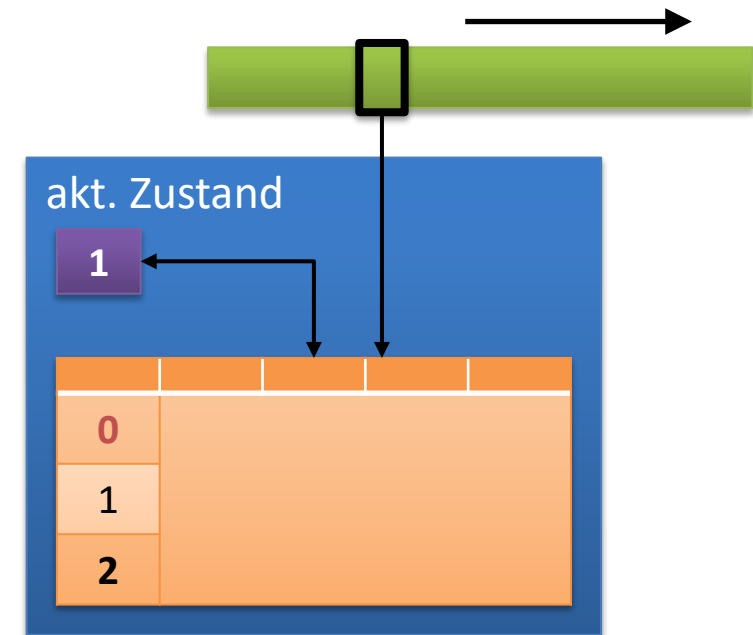
Die Markierung der Kanten stammt dabei aus der Menge T .

Die von dem Übergangsdiagramm akzeptierte Sprache ist

$$L(UD) = \{ w \in T^* \mid \text{es gibt einen } w\text{-Weg von } v_0 \text{ zu einem } v_f \in V_f \text{ in } UD \}$$

Nichtdeterministischer endlicher Automat (NEA)

- NEA ist ein zu Übergangsdiagrammen äquivalenter Mechanismus
- Ein **NEA** besteht aus:
 - einer **Variablen**, die nur endlich viele verschiedene Zustände annehmen kann;
 - einem **Lesekopf**, mit dem das **Eingabeband** von links nach rechts gelesen wird;
 - einer **Übergangsrelation**, die die Kontrolle des Automaten bildet;
 - einem **Anfangszustand** und
 - ein oder mehreren **Endzuständen**.



- Definition
- Rechnung

Rechnung eines NEA

Definition: NEA

- Ein **(nichtdeterministischer) endlicher Automat (NEA)** ist ein Tupel

$$M = (\Sigma, Q, \Delta, q_0, F),$$

mit

- Σ ist endliches Alphabet (das **Eingabealphabet**)
- Q ist endliche Menge von **Zuständen**
- $q_0 \in Q$ ist **Anfangszustand**
- $F \subseteq Q$ ist Menge der **Endzustände**
- $\Delta \subseteq Q \times (\square \cup \{\varepsilon\}) \times Q$ ist **Übergangsrelation**

Rechnung eines NEA

- Konfigurationen: $K = Q \times \Sigma^*$

- Rechnung:

$$(q, a.w) \rightarrow (p, w) \quad \Leftrightarrow \quad (q, a, p) \in \Delta$$

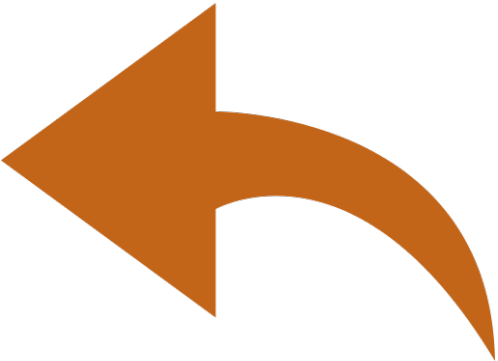
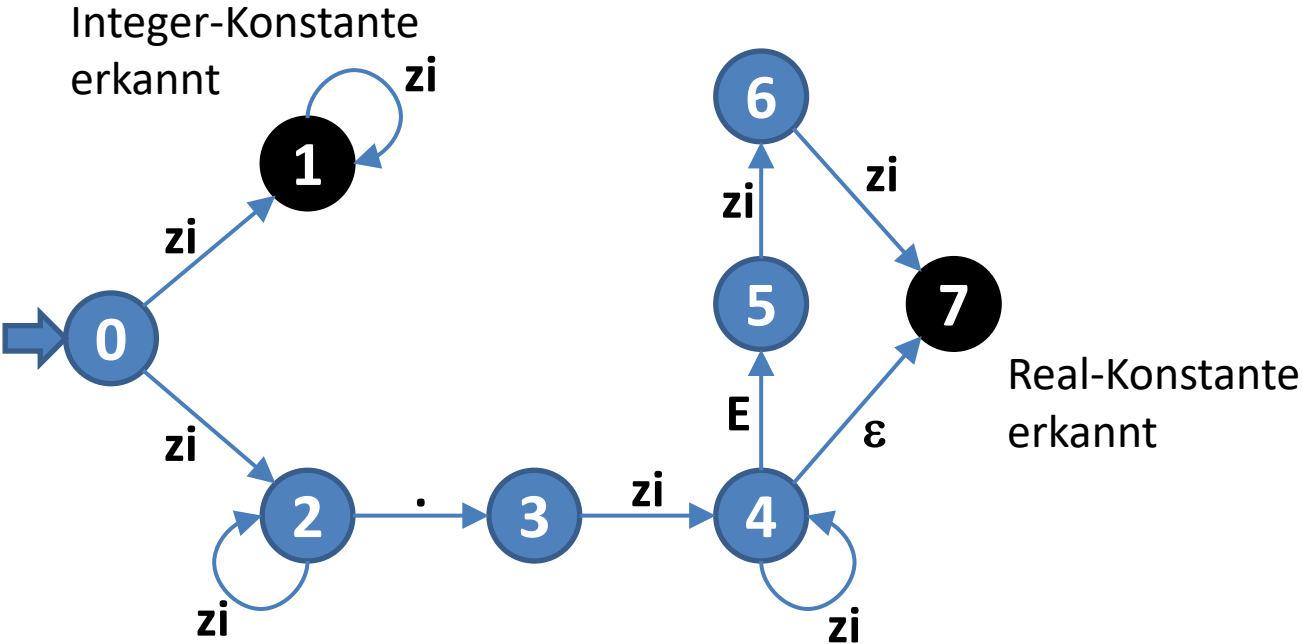
$$(q, w) \rightarrow (p, w) \quad \Leftrightarrow \quad (q, \varepsilon, p) \in \Delta$$

$\rightarrow^*$ ist die reflexive, transitive Hülle von $\rightarrow$

- Die vom NEA akzeptierte Sprache:

$$L(NEA) = \{w \mid (q_0, w) \rightarrow^* (p, \varepsilon) \text{ und } p \in F\}$$

Korrespondenz zwischen Übergangsdiagramm und NEA



NEA	Übergangsdiagramm
$q \in Q$	
q_0	
$q_f \in F$	
$(q,a,p) \in \Delta$	

Zugehöriger NEA

- Alphabet $\Sigma = \{ 0, 1, \dots, 9, ., E \}$
- Menge von Zuständen $Q = \{ z_0, z_1, \dots, z_7 \}$
- Anfangszustand $q_0 = z_0$
- Endzustände $F = \{ z_1, z_7 \}$
- Übergangsrelation $\Delta \subseteq Q \times (\Sigma \cup \{ \varepsilon \}) \times Q$:

Eingabe: 123.45E12

$(z_0, 1, z_1)$	$(z_0, 1, z_2)$	
$(z_1, 2, z_1)$	$(z_2, 2, z_2)$	
$(z_1, 3, z_1)$	$(z_2, 3, z_2)$	
kein	$(z_2, ., z_3)$	
Übergang	$(z_3, 4, z_4)$	
	$(z_4, 5, z_4)$	(z_4, ε, z_7)
	(z_4, E, z_5)	kein Übergang
	$(z_5, 1, z_6)$	(z_4, ε, z_7)
	$(z_6, 2, z_7)$	kein Übergang

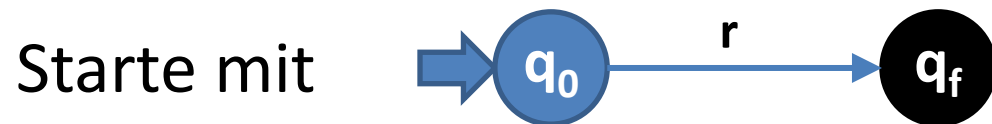
	{ 0, 1, ..., 9 }	.	E	ε
z_0	{ z_1, z_2 }	$\emptyset$	$\emptyset$	$\emptyset$
z_1	{ z_1 }	$\emptyset$	$\emptyset$	$\emptyset$
z_2	{ z_2 }	{ z_3 }	$\emptyset$	$\emptyset$
z_3	{ z_4 }	$\emptyset$	$\emptyset$	$\emptyset$
z_4	{ z_4 }	$\emptyset$	{ z_5 }	{ z_7 }
z_5	{ z_6 }	$\emptyset$	$\emptyset$	$\emptyset$
z_6	{ z_7 }	$\emptyset$	$\emptyset$	$\emptyset$
z_7	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$

RA $\rightarrow$ NEA

Satz:

Zu jedem regulären Ausdruck r gibt es einen nichtdeterministischen endlichen Automaten, der die von r beschriebene reguläre Menge akzeptiert.

Der Beweis dieses Satzes ist konstruktiv und wird durch den Algorithmus RA (reguläre Ausdrücke) nach NEA beschrieben:

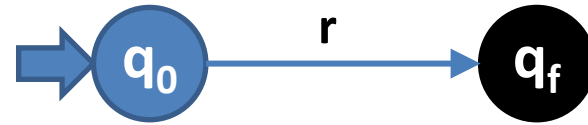


Zerlege die regulären Ausdrücke an den Kanten und verfeinere das Übergangsdiagramm, bis alle Kanten mit Zeichen aus Σ oder ε markiert sind.

Eingabe: regulärer Ausdruck r über Σ

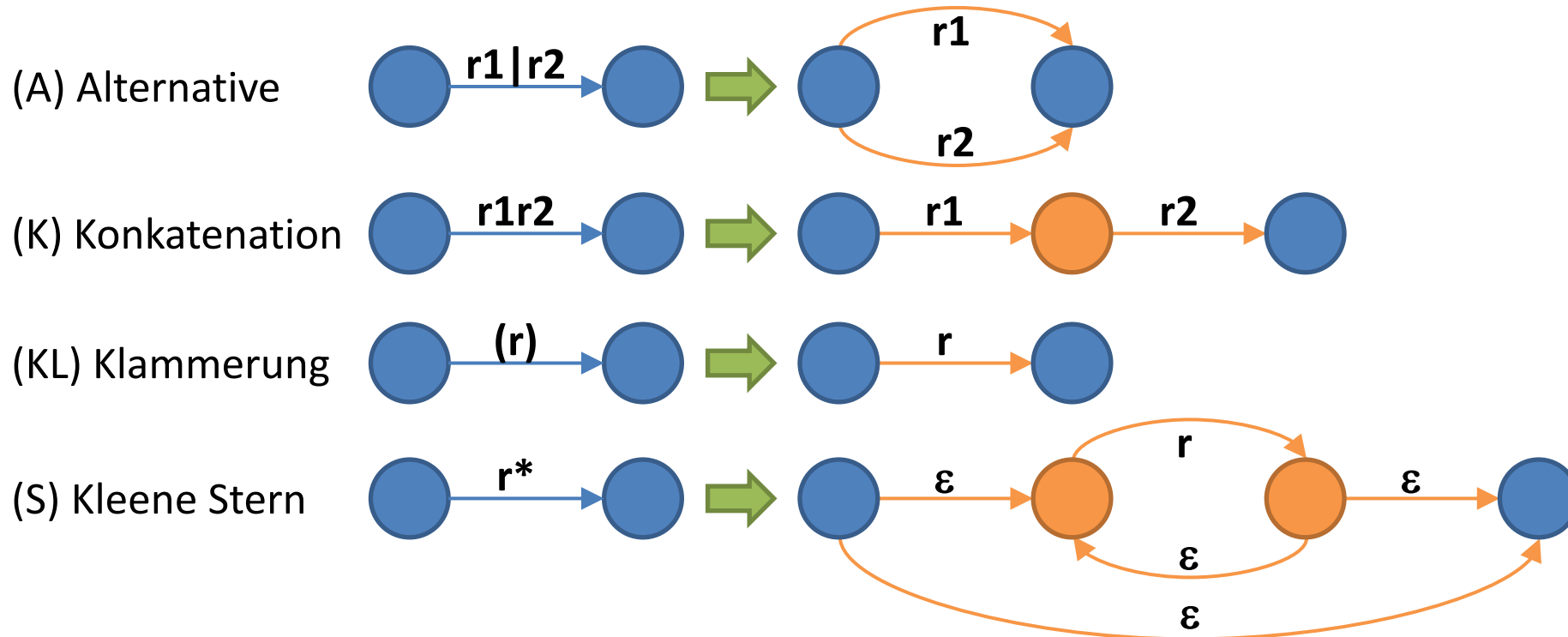
Ausgabe: Übergangsdiagramm eines NEA

Methode: Start



Algorithmus:
 $RA \rightarrow NEA$

Wende folgende Regeln so lange auf das jeweilige Ausgangsdiagramm an, bis alle Kanten mit Zeichen aus Σ oder ε markiert sind. Die Knoten in den linken Seiten der Regeln werden identifiziert mit Knoten im aktuellen Übergangsdiagramm. Alle in der rechten Seite einer Regel **neu auftretenden Knoten** entsprechen neu kreierten Knoten, also neuen Zuständen.



$$a \mid (bc)^*$$

Deterministischer endlicher Automat (DEA)

- **Definition: DEA**

Sei $M = (\Sigma, Q, \Delta, q_0, F)$ ein NEA. M heißt **deterministischer endlicher Automat (DEA)**, wenn Δ eine Funktion $\delta : Q \times \Sigma \rightarrow Q$ ist.

- **Bemerkungen:**

- Ein DEA hat keine ε -Übergänge.
- Für jedes Paar (q, a) mit $q \in Q, a \in \Sigma$ gibt es **höchstens** einen Nachfolgezustand.
- ➡ Es gibt maximal eine akzeptierende Rechnung für jedes Wort w
- ➡ Im Gegensatz zum NEA muss ein DEA nicht „raten“ und ist daher geeigneter für die lexikalische Analyse.

Sollbruchstelle



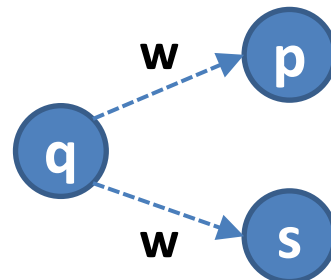
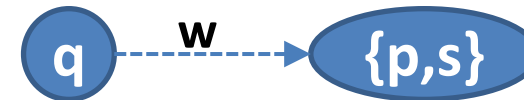
Alles ist eine Menge

NEA $\rightarrow$ DEA

Satz:

Wird eine Sprache L von einem NEA akzeptiert, so gibt es einen DEA, der L akzeptiert.

Der Beweis ist konstruktiv und benutzt die so genannte **Teilmengekonstruktion**. Gibt es zwei w -Wege, so fallen die entsprechenden Zustände beim DEA in einen gemeinsamen Zustand.

NEA:**DEA:**

Ein w -Weg ist ein Weg von einem Knoten q zu einem Knoten p , so dass die Konkatenation der Knotenmarkierungen das Wort w ergibt.

Definition: ε -Folgezustände

- Sei $M = (\Sigma, Q, \Delta, q_0, F)$ ein NEA und sei $q \in Q$. Die Menge der ε -Folgezustände von q ist:

$$\varepsilon_{FZ}(q) = \{ p \mid (q, \varepsilon) \rightarrow^* (p, \varepsilon) \}$$

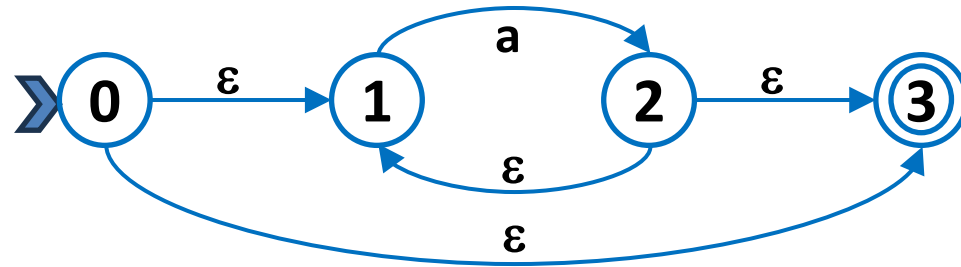
also die Menge aller Zustände p , inklusive q , für die es im Übergangsdiagramm zu M einen ε -Weg von q nach p gibt. Wir erweitern ε_{FZ} auf Mengen von Zuständen $S \subseteq Q$:

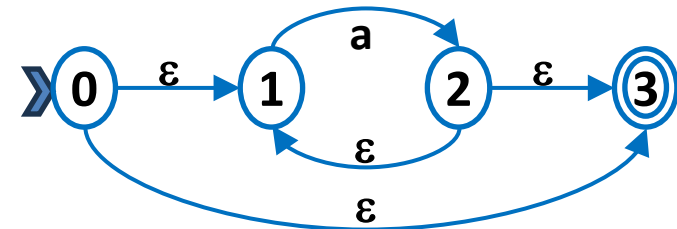
$$\varepsilon_{FZ}(S) = \bigcup_{q \in S} \varepsilon_{FZ}(q)$$

Definition: Zu einem NEA gehörender DEA

- Sei $M = (\Sigma, Q, \Delta, q_0, F)$ ein NEA. Der zu M **gehörende DEA** $M'' = (\Sigma, Q'', \delta, q_0'', F'')$ ist definiert durch:
 - $Q'' = \mathcal{P}(Q) = \{S \mid S \subseteq Q\}$, die Potenzmenge von Q
 - $q_0'' = \varepsilon_{FZ}(q_0)$
 - $F'' = \{S \subseteq Q \mid S \cap F \neq \emptyset\}$
 - $\delta(S, a) = \varepsilon_{FZ}(\{p \mid (q, a, p) \in \Delta \text{ für } q \in S\})$ für $a \in \Sigma$

NEA für a^*



$\{0\} \cdot$ $\{0,1\} \cdot$ $\{0,2\} \cdot$ $\cdot\{0,3\}$ $\{1\} \cdot$ $\cdot\{1,2\}$ $\{1,3\} \cdot$ $\cdot\{2\}$ $\{2,3\} \cdot$ $\cdot\{3\}$ $\{0,1,2\} \cdot$ $\cdot\{0,1,3\}$ $\{0,2,3\} \cdot$ $\{1,2,3\} \cdot$ $\{0,1,2,3\} \cdot$ $\cdot\{\}$  a a 

Eingabe: NEA $M = (\Sigma, Q, \Delta, q_0, F)$

Ausgabe: DEA $M'' = (\Sigma, Q'', \delta, q_0'', F'')$

Algorithmus:

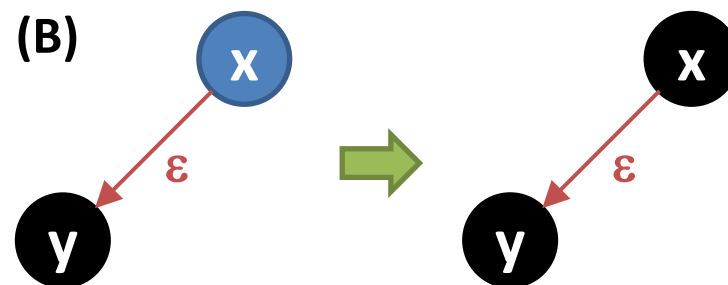
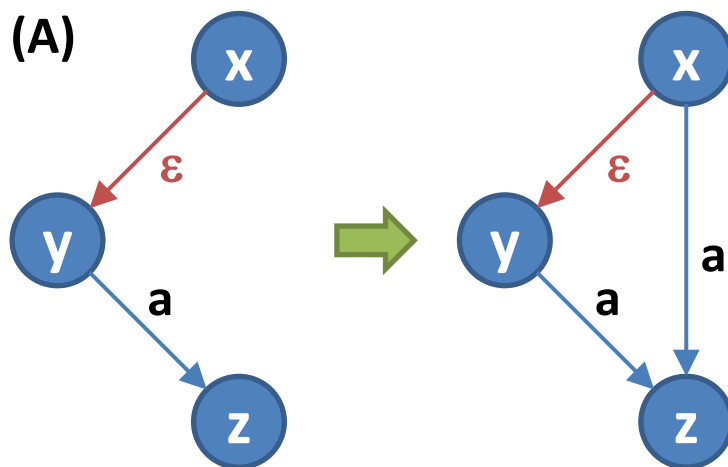
NEA $\rightarrow$ DEA

Schritt 1: Entfernen der ε -Kanten

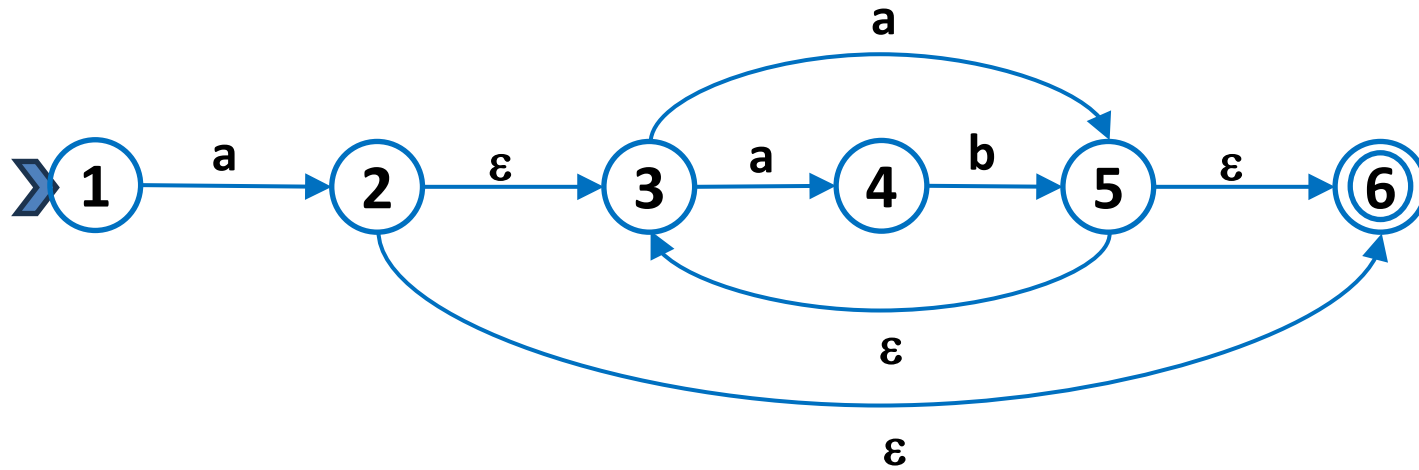
Methode: Schritt 1: Entfernen der ε -Kanten.

Wende die Regeln A und B so lange auf das jeweilige Ausgangsdiagramm an, bis alle ε -Kanten aus dem Diagramm entfernt sind. Die Kanten in den linken Seiten der Regeln werden identifiziert mit Kanten im aktuellen Übergangsdiagramm. Alle in der rechten Seite einer Regel neu auftretenden Kanten entsprechen neu kreierten Kanten, also neuen Übergängen.

Nach Anwendung aller, auf eine ε -Kante anwendbaren Regeln, wird diese entfernt. Der nach diesen Regeln entstandene NEA ohne ε -Kanten sei: $M' = (\Sigma, Q', \Delta', q_0', F')$



ε -Kanten aus NEA für $a(a|ab)^*$ entfernen



Algorithmus:

NEA $\rightarrow$ DEA

Schritt 2: NEA ohne ε -Kanten $\rightarrow$ DEA

Schritt 2: NEA M' ohne ε -Kanten $\rightarrow$ DEA M'' .

Zustände in M'' sind Mengen von Zuständen von M' . Startzustand von M'' ist $\varepsilon_{FZ}(q_0')$. Die zu Q'' hinzuerzeugten Zustände werden markiert, sobald ihre Folgezustände bzw. Übergänge unter allen Symbolen aus Σ erzeugt worden sind. Die Markierung von erzeugten Zuständen wird in der Funktion

$$\textit{marked} : P(Q) \rightarrow \textit{bool}$$

festgehalten.

Algorithmus:

NEA $\rightarrow$ DEASchritt 2: NEA ohne ε -Kanten $\rightarrow$ DEA $q_0'' := \{q_0'\};$ $Q'' := \{q_0''\};$ $\text{marked}(q_0'') := \text{false};$ $\delta := \emptyset;$ **while** (existiert $S \in Q''$ and $\text{marked}(S) = \text{false}$) **do** $\text{marked}(S) := \text{true};$ **foreach** $a \in \Sigma$ **do** $T := \{p \in Q' \mid (q, a, p) \in \Delta' \text{ and } q \in S\};$ **if** $T \notin Q''$ **then** $Q'' := Q'' \cup \{T\};$ $\text{marked}(T) := \text{false};$ **fi**; $\delta := \delta \cup \{(S, a) \rightarrow T\};$ **od**;**od**; $F'' := \{S \subseteq Q'' \mid S \cap F' \neq \emptyset\}$

DEA für $a(a|ab)^*$

Minimierung eines DEA

- Die erzeugten deterministischen endlichen Automaten sind i.a. nicht die kleinstmöglichen, die die Ausgangsprache akzeptieren. Es kann nämlich noch Zustände mit gleichem **Akzeptanzverhalten** geben.
- Zwei Zustände p und q haben gleiches Akzeptanzverhalten, wenn der Automat aus p und q unter genau den gleichen Eingabewörtern w einen Endzustand erreicht:

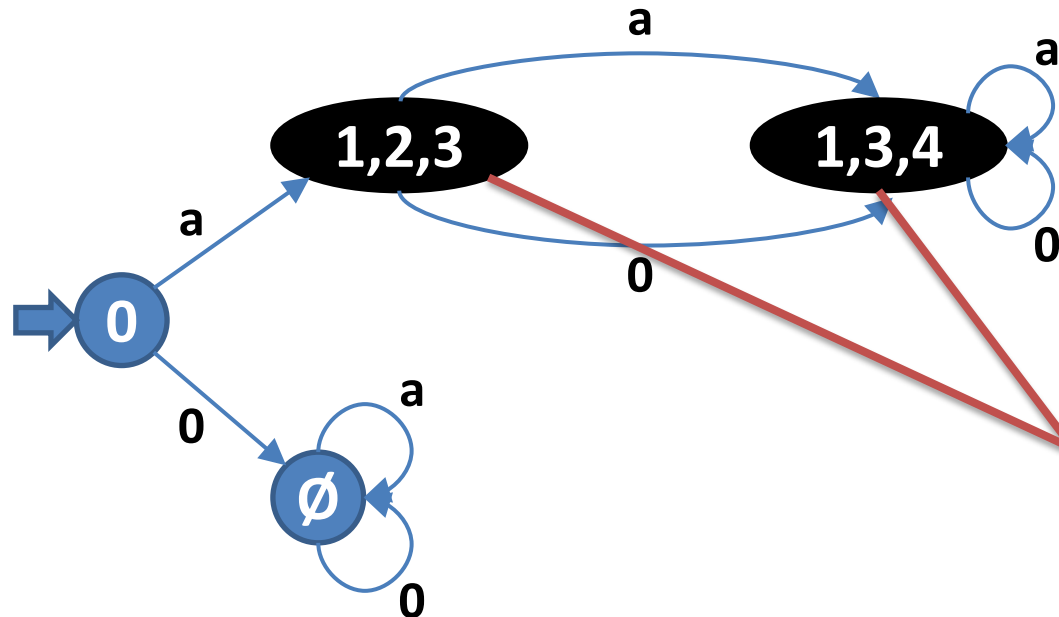
$$\forall w : \delta(p, w) \in F \leftrightarrow \delta(q, w) \in F$$

Notation:

$$\delta(p, w) = q$$

$\leftrightarrow$

$$(p, w) \rightarrow^* (q, \varepsilon)$$



**identisches
Akzeptanzverhalten**
(=erkennt gleichen Suffix)

Algorithmus: MinDEA

Eingabe: DEA $M = (\Sigma, Q, \delta, q_0, F)$

Ausgabe: DEA $M' = (\Sigma, Q', \delta', q_0, F')$ minimal

Methode:

Schritt 1: Nicht erreichbare Zustände entfernen. *(Nicht notwendig, wenn der angegebene NEA $\rightarrow$ DEA Algorithmus vorher verwendet wurde)*

Alle Zustände, die nicht vom Startzustand aus über eine beliebige Menge von Übergängen des DEA M erreichbar sind, werden entfernt

Das ist eine Menge, kein Tupel !

Schritt 2: Markierungstabelle erstellen.

1. Aufstellen einer Tabelle aller Zustandspaare $\{q, q'\}$ mit $q \neq q'$ von M .
2. Markiere alle Paare $\{q, q'\}$ mit $q \in F$ und $q' \notin F$.
3. Für jedes *unmarkierte* Paar $\{q, q'\}$ und jedes $a \in \Sigma$ teste, ob $\{\delta(q, a), \delta(q', a)\}$ bereits markiert ist. Wenn ja: markiere auch $\{q, q'\}$.
4. Wiederhole den letzten Schritt, bis sich keine Änderung in der Tabelle mehr ergibt.

Schritt 3: Zustände vereinigen.

Vereinige alle *unmarkierten* Zustandspaare $\{q, q'\}$ nach folgenden Regeln:

- (1) falls $\delta(p, a) = q'$ mit $p \in Q, a \in \Sigma$ dann $\delta := \delta \cup \{(p, a) \rightarrow q\}$
- (2) falls $\delta(q', a) = p$ mit $p \in Q, p \neq q', a \in \Sigma$ dann $\delta := \delta \cup \{(q, a) \rightarrow p\}$
- (3) entferne q' aus Q
- (4) entferne $(p, a) \rightarrow q'$ und $(q', a) \rightarrow p$ aus δ für alle $p \in Q, a \in \Sigma$

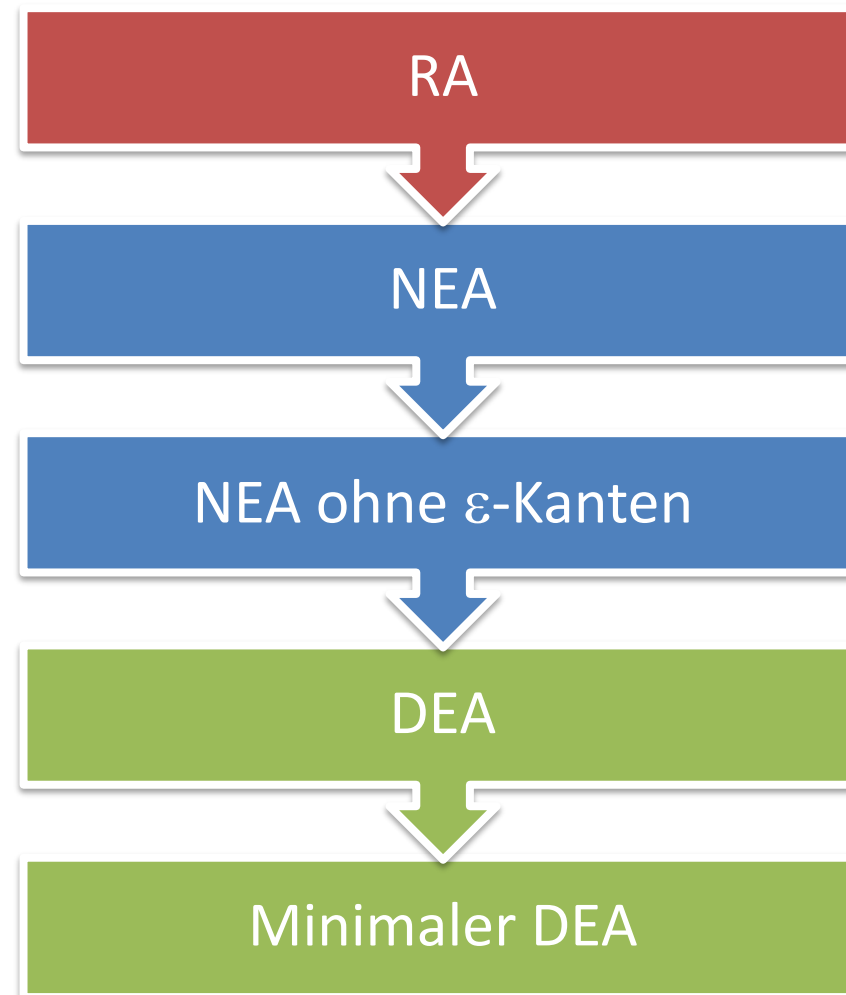
Idee des Algorithmus:

Endzustände und Nichtendzustände haben offensichtlich unterschiedliches Akzeptanzverhalten. Propagiere diese Information rückwärts. Danach vereinige Zustände mit gleichem Verhalten.

Minimaler DEA für $a(a|ab)^*$

	1	2	3	4	5
1					
2					
3					
4					
5					

Vom regulären Ausdruck zum minimalen deterministischen endlichen Automaten



Scannergeneratoren

JFlex/Lex

